

TASK 8: Interfaces & Abstraction – Payment Gateway Simulator

Tools:

- Primary: IntelliJ IDEA
- Alternatives: Eclipse

Hints / Mini Guide:

1. Design a Payment interface.
2. Implement multiple payment modes.
3. Use abstraction to hide logic.
4. Apply loose coupling.
5. Demonstrate multiple inheritance using interfaces.
6. Handle invalid payment cases.
7. Switch payment modes dynamically.
8. Add meaningful logs.

Deliverables:

- Payment simulation program
- Clean interface design

Interview Questions Related To Above Task:

- Interface vs abstract class?
- Why abstraction is important?
- Can interface have methods?
- Multiple inheritance in Java?
- Real-world use of interfaces?

📌 Task Submission Guidelines

- 🕒 **Time Window:**

You can complete the task anytime between 10:00 AM to 10:00 PM on the given day. Submission link closes at 10:00 PM

- 🔍 **Self-Research Allowed:**

You are free to explore, Google, or refer to tutorials to understand concepts and complete the task effectively.

- 🔧 **Debug Yourself:**

Try to resolve all errors by yourself. This helps you learn problem-solving and ensures you don't face the same issues in future tasks.

- 💰 **No Paid Tools:**

If the task involves any paid software/tools, do not purchase anything. Just learn the process or find free alternatives.

- 📁 **GitHub Submission:**

Create a new GitHub repository for each task.

Add everything you used for the task — code, datasets, screenshots (if any), and a short README.md explaining what you did.

- 📌 **Submit Here:**

After completing the task, paste your GitHub repo link and submit it using the link below:

- 👉 [[Submission Link](#)]

Best
of
Luck

